```
#include<stdio.h>
#include<stdlib.h>

struct node{
   int data;
   struct node *prev;
   struct node *next;
};

struct node *head = NULL;

void insert_begin(){
   struct node *newnode;
   newnode=(struct node*)malloc(sizeof(struct node));

   printf("\nENTER DATA= ");
   scanf("%d",&newnode->data);

   newnode->prev=NULL;
   newnode->next=head;

   if(head!=NULL)
      head->prev=newnode;

   head=newnode;
}

void insert_end(){
   struct node *newnode,*temp;

   newnode=(struct node*)malloc(sizeof(struct node));

   printf("\nENTER DATA= ");
   scanf("%d",&newnode->data);

   newnode->next=NULL;
```

```
    newnode->prev=NULL;

    if(head==NULL){
      head=newnode;
    }
    else{
      temp=head;

      while(temp->next!=NULL){
        temp=temp->next;
      }

      temp->next=newnode;
      newnode->prev=temp;
    }
}

void display_forward(){
    struct node *temp=head;

    if(head==NULL){
      printf("\nLIST IS EMPTY");
    }

    printf("\n-----FORWARD-----\n");

    while(temp!=NULL){
      printf("%d ",temp->data);
      temp=temp->next;
    }

    printf("\n");
}
void delete_node()
{
    struct node *temp;
```

```
    int value;

    if(head == NULL)
    {
        printf("LIST IS EMPTY \n");
        return;
    }

    printf("ENTER YOUR VALUE TOP DELETE= ");
    scanf("%d",&value);

    temp=head;

    while(temp != NULL && temp->data != value)
        temp = temp->next;

    if(temp == NULL)
    {
        printf("NODE NOT FOUND\n");
        return;
    }

    if(temp->prev != NULL)
        temp->prev->next=temp->next;
    else
        head = temp->next;

    if(temp->next != NULL)
        temp->next->prev=temp->prev;

    free(temp);
    printf("NODE DELETED SUCCESSFULLY\n");
}

void display_reverse()
{
```

```
    struct node *temp = head;

    if(head == NULL)
    {
        printf("LIST IS EMPTY\n");
        return;
    }

    while(temp->next != NULL)
        temp=temp->next;

    printf("---BACKWARD---\n");

    while(temp != NULL)
    {
        printf("%d ",temp->data);
        temp=temp->prev;
    }

    printf("\n");
}



int main()
{
    int choice;
    do{
        printf("\n------ DOUBLY LINKED LIST ------\n");
        printf("\n1.INSERT AT BEGINNING\n");
        printf("\n2.INSERT AT END\n");
        printf("\n3.DISPLAY FORWARD\n");
        printf("\n4.DELETE NODE\n");
        printf("\n5.DISPLAY REVERSE\n");
        printf("\n6.EXIT\n");
        printf("\nENTER YOUR CHOICE= ");
```

```
    scanf("%d",&choice);

    switch (choice)
    {
      case 1:
        insert_begin();
        break;

      case 2:
        insert_end();
        break;

      case 3:
        display_forward();
        break;

      case 4:
        delete_node();
        break;

      case 5:
        display_reverse();
        break;

      case 6:
        printf("\nEXITING PROGRAM \n");
        break;

      default:
        printf("\nINVALID CHOICE");
    }
  }
  while (choice!=6);

  return 0;
}
```